Intro to Data Structures

Data Structures

- What is a data structure?
  - A method of storing, accessing, and manipulating data
- Examples:
  - Linked Lists
  - Hash maps
  - Red-black trees
  - Stacks and queues
  - Sets
  - Etc.

Terminology

- Data Structure vs abstract data type (ADT)
  - ADT – the logical operation of a type (user's perspective)
  - Data Structure – an implemented ADT

Basic Data Structures - Arrays

- Simplest data structure: Arrays
  - linear collection of items
  - typically fixed size and contiguous
  - built into every language

Basic Data Structures - Arrays

- Common operations
  - get item
  - set item
  - get length
  - clear, isEmpty, find

Flexibility?

- Allow for dynamic sizes
  - Vectors
  - Lists

Vectors

- what if arrays cold grow?
- We need to track the current size of the array, and how many items we have
- What happens when we need to add an item while the array is full?

Vectors - continued

- REALLOCATE!!!

Vectors – still…

- Pros
  - Dynamic sizing
  - Contiguous memory
  - We still have random access (by index)
- Cons
  - Requires some book keeping
  - Contiguous memory
  - Memory allocations

Lists

- Typically referred to as linked lists
- Singly or doubly linked

Lists - continued

- Pros
  - Dynamic sizing
  - Non-contiguous memory
  - Certain operations are very fast
- Cons
  - Requires many pointers
  - Non-contiguous memory
  - Constantly traversing, no random access

Bigger Picture

- Vectors and lists do similar things
- We can consider them dynamic arrays
- Operations
  - Append
  - Insert
  - Remove, pop
  - Find, has
  - getIndex

Stacks

- Stacks are an abstract data type
- Operations
  - Push
  - Pop
  - isEmpty
- Stacks are LIFO – Last in, First Out

Stacks - continued

- Stacks are an ADT, how would we implement them?
  - Arrays?
  - Vector?
  - Linked List?
- Use
  - Undo
  - Checking that parentheses match
  - Call stack!!!

Queues

- Stacks, but in reverse
- Queues are FIFO – First in, first out
- Operations
  - Push (enqueue)
  - Pop (dequeue)